REBEL LEADERS / FIELD TOOL 01



AI Stalemate

THE LEAD DEVELOPER FIELD GUIDE

Make your judgment count. Shape responsible use with your PM.

Nic Payne

For you, the lead developer

OPEN YOUR LEAD DEVELOPER LAB >

UNDERSTAND THE PRESSURE. WORK THE SITUATION. LEAVE WITH A MOVE.

WORKING EDITION / REBEL-LEADER.COM

Your PM sees potential. You own what ships.

Your PM wants the team to use AI. You may see promise, but you also see the verification, review, and maintenance it creates. This is a way to make your judgment useful and help shape the opportunity.

Get specific about the outcome

Ask your PM what they want to improve for customers or the team. Is the problem a customer wait, repeated work, missed learning, or constrained capacity? A clear outcome gives you something to evaluate.

Name the cost you actually carry

Point to the work or risk in this use: acceptance checks, missing codebase context, data boundaries, reviewer effort, or maintenance. Make your concern concrete enough to change the proposal.

Check your story about the request

Your PM asked for a trial by Friday. That is an event. They do not care about quality is an interpretation. Ask what is driving the request before treating your interpretation as their intent.

Help author the approach

Try: I can see why that outcome matters. Here is the use I would consider, the quality bar I would keep, and the capacity we would need. Your technical judgment can shape a better proposal.

> Your win: help the team learn where AI earns its place, protect the standards you own, and make a decision others can trust.

Paint a future worth working toward.

Get clear about the future you would want to help build. You can lead responsible adoption by connecting engineering judgment to a customer and team outcome your PM cares about.

Name the improvement you care about

Which improvement would be worth your engineering attention? Less repetitive work, more time for design, faster reliable feedback, or practices other developers can trust? Say why you would invest in it.

Paint a recognizable before and after

Describe the future in everyday work: shorter customer waits, fewer repetitive tasks, more time for judgment, or patterns other teams can trust. Make the picture specific enough to question and improve.

Connect it to a shared stake

The company could gain a useful capability. The team could gain mastery and influence over its standards. Building something others trust could also create professional opportunities for both leaders.

Invite authorship

Ask your PM: Which outcome matters most? What would you change in this picture? Co-author the ambition and ask them to help protect the time needed to pursue it responsibly.

Connect the picture to the next move

A vision gives the work direction. The next move makes it credible: clarify a concern, resolve permission or capacity, use existing evidence, or agree on one bounded test.

> Try: I'd like us to find the uses that actually improve accepted work. If we can make those practices trustworthy, we could help set the company's standard and give our team more room for the hard problems. That could create opportunities for both of us. Which outcome would make this worth pursuing from your side?

Make the concern workable.

Make your concerns useful to the PM. Describe where the proposed use breaks down, what condition could make it workable, or why the evidence already supports stopping. Ask for the priority tradeoff when capacity is the issue.

The quality is not good enough

Which checks or standards fail? Consider a smaller task, clearer acceptance criteria, repository conventions, automated tests, and human review. Keep the existing quality bar.

Reviewing it takes longer

Where does effort increase? Consider smaller changes, explicit author verification, and protected review time. Measure accepted work across the whole workflow rather than the first draft.

Our data or code cannot go into that tool

Which inputs and approvals are involved? Establish the permitted tool and input boundary with the approval owner. Synthetic or non-sensitive inputs may be an option; permission comes first.

It does not understand our codebase

Which context or conventions are missing? Consider explicit repository instructions, a known example, and a contained task. Check whether the available context actually supports that use.

We do not have capacity

What work would this replace, and who must review it? Ask the priority owner to protect time or decline the investment. Unplanned overtime is not the trial budget.

We tried it and it was worse

Which task, conditions, and result? Use that evidence. If a materially different use is proposed, name what changed and why another test would answer a useful question.

GitHub: Copilot best practices

Thoughtworks: reducing AI development friction

> The useful question is: Which condition would make this proposal workable, and who can decide it? Keep a specific no and an unsupported yes open to the same scrutiny.

Let's get ready to talk with your PM.

Take 5-10 minutes before you meet your PM. Write the specific work you would protect, the outcome you could support, and what evidence would change your view.

What conversations have you and your PM already had, and where do things stand?

Use the request, words, or event you can describe. Be precise enough that the other person could recognize it.

What are you assuming about your PM's motives?

Name your interpretation of their motive. Hold it as a hypothesis rather than a fact.

What quality, review, or maintenance responsibility are you protecting?

Describe the opportunity, cost, obligation, or outcome you carry. Product and Engineering can both have legitimate answers.

What have you observed in the actual workflow?

Separate a concrete past result from a prediction about future use. An observed failure deserves attention; it does not settle every use case.

What evidence would change your assessment of this use?

Name the evidence that could alter your view. A trial is not useful if neither side would reconsider anything.

What do you need to ask your PM about the outcome or priority?

Leave room for their actual account of the cost. Do not turn private preparation into a better argument for your original position.

> You can reflect without typing. Any private notes you type stay only in this open tab: they are not saved by the Lab or included in shared exports. Refreshing or leaving the Lab clears them.

Ask what your PM is hearing.

Your PM may hear costs of delay, customer commitments, and opportunities you haven't seen. Ask for the context behind the urgency, then help turn it into a credible investment.

Name the outcome and the consequence

Ask which outcome matters, who is affected by delay, and what timing changes. If you hear $500,000 per week, ask where the estimate comes from and which assumptions it depends on.

Keep estimates open to examination

Separate an observed cost, a forecast, and a stakeholder's claim. Name who can validate the figure. Urgency helps prioritize an investment; the engineering and access requirements still need owners.

Make the opportunity shared

A team that establishes trustworthy practices could improve delivery and influence how the company works. Connect that possibility to learning, mastery, customer value, and professional opportunity for both leaders.

Make the investment explicit

Translate the concern into an investment your PM can discuss: an access-model decision, a qualified reviewer, training, or a smaller use. Explain what that investment would enable.

> Try together: The opportunity matters. Let's put the business case and the conditions for responsible delivery in one picture.

Make access a design decision.

Make the specific access gap visible to your PM. Name the route, action, or data boundary at risk and the design decision and verification it needs.

Specify who can do what to which data

Map users or groups to application roles, allowed actions, and resource boundaries. Include joiners, movers, leavers, and ownership of access changes. New apps need a coherent access model across the relevant systems.

Verify permissions in the API

Microsoft Entra sign-in establishes identity. APIs also need the appropriate token, scope, role, and resource-level authorization checks. Hiding a button does not protect a route. Include tests where access must be denied.

Bound what an agent can do

Record which environment and data it can use, which credentials it receives, and whether it can read, write, run commands, or deploy. Name approval points, least-privilege boundaries, and a recovery owner.

Assign the decision and verification

Name the access-policy owner, engineering/security reviewer, required checks, and evidence of completion. Use the Lab to coordinate the work; verify implementation through the appropriate technical review.

Microsoft: app roles and groups

Microsoft: verify API scopes and roles

OWASP: authorization checks

> A real prerequisite can change the next move: resolve the access model, restrict the environment, or pause this use until the required verification exists.

Match the responsibility with the capability.

Assess the work your team can explain and verify today. Make the support required for this use visible to your PM, with a concrete readiness check.

Start with demonstrated capability

Can the developer explain the change, test permission boundaries, review a diff, debug a failure, and describe deployment and rollback? Prior roles and age do not establish these capabilities; practical evidence does.

Protect learning and qualified review

Use pairing, focused training, small changes, and a reviewer qualified for the risk. Ask developers to explain why the generated change is correct and how they checked it. Reserve time for learning and review.

Build a repeatable delivery practice

Clarify acceptance criteria, Git and review practices, testing, deployment ownership, and feedback. Scrum familiarity helps a team coordinate; secure engineering capability also needs deliberate practice and coaching.

Treat acceleration as conditional

DORA describes AI as amplifying the strengths and weaknesses of the surrounding delivery system. Small batches strengthen feedback and review. Record what this team needs for this use, then evaluate accepted work at its full cost.

NIST: Secure Software Development Framework

GitHub: responsible use of Copilot agents

DORA: working in small batches

DORA: AI-assisted development research

> Prepare first can be a productive decision. Name the training, pairing, review, or platform work, the owner, and the evidence that will show the team is ready.

One test you could both believe.

Spend about 30 minutes with your PM shaping one test you could trust. You bring technical suitability and acceptance checks; your PM helps clarify the outcome and the priority tradeoff.

Agree on the ambition

Share the future you hope to help create and why it matters. Ask the other leader to shape it. Record only the ambition you both agree to pursue in shared work.

Bound the work

Name the work outcome, exact task, and work that is out of scope. Choose a comparator without AI and say why the task size, experience, and acceptance bar are comparable.

Set the permission boundary

Confirm the approved tool, model/version, allowed inputs, and approval owner. If permission is missing, name exactly what must be resolved before work can start.

Protect quality and capacity

Define the acceptance checks and accountable engineering owner. Estimate and protect author and reviewer time. Ask your PM to make the displaced work visible to the priority owner.

Agree on the bar beforehand

Define success across the full workflow, not just generation speed. Write a stop rule that can be acted on during the trial.

Make ownership explicit

Name the task owner, reviewer, decision owner, and review date. Export the Trial Card now, before the test. If a prerequisite is missing, name it and choose PAUSE.

> A good trial: one approved unit-test task with comparable work and reserved review time. A weak trial: Everyone use AI this week and report whether it felt faster.

Count the work on both sides.

Record the work you and the reviewer actually do, including checks, rework, and later defects. Give your PM a full picture of cost and quality that they can responsibly communicate.

Workflow stage	Without AI	With AI
Setup / context	10 min	15 min
Drafting	45 min	12 min
Author verification	15 min	25 min
Reviewer checking	15 min	30 min
Rework	5 min	15 min
Follow-up	0 min	0 min
Total person-minutes	**90 min**	**97 min**

A separate effort example

These fictional measurements describe one matched unit-test task. The acceptance bar passed after rework. Drafting was faster with AI, while verification, review, and rework increased.

What the totals establish

Drafting took 33 fewer minutes, but the full recorded workflow took 7 more. Reviewer checking increased by 15 minutes. The agreed effort bar was missed in this local test.

What remains outside the totals

Record acceptance checks, defects, reviewer experience, tool cost, elapsed time, first-use learning, and limits to comparability. Name an owner and date for a later defect check.

> A complete total is useful evidence. It is not a universal AI productivity claim, a quality verdict, or proof of causation.

Four legitimate next moves.

Review the evidence with your PM and the agreed decision owner. Be specific about what this use supports, what would need to change, and when another test would be worthwhile.

CONTINUE

The evidence supports this bounded use under these conditions. Record the operating boundaries and the next review. Continue does not mean AI everywhere.

REVISE

The use may have value, but the task, inputs, review workflow, or boundaries need to change. Name what will change before another approved trial.

STOP

The evidence does not support this use, or a guardrail was breached. Record why and preserve what was learned. There is no obligation to turn a failed use case into adoption.

PAUSE

A prerequisite, approval, owner, capacity, or useful evidence is missing. Name the prerequisite and who will resolve it. Pause can be chosen before a trial starts.

> Every decision needs a reason, the limits of the evidence, an accountable next-action owner, and a date. Record unresolved disagreement rather than claiming consensus.

A worthwhile opportunity. Preparation first.

Fictional case: your PM hears a costly delay from the sponsor. You identify unverified API permissions and gaps in the team's engineering practice. Together, you define what responsible progress requires.

The business context

The sponsor estimates $500,000 per week in operational opportunity. The figure is not yet validated. The PM will clarify the assumptions with Finance. The team sees why the outcome matters.

The system and team prerequisites

Access roles and record boundaries across apps are unresolved. Routes need allowed/denied-access tests. Developers need pairing, qualified review, and practice explaining, testing, deploying, and rolling back a change.

The Shared Next-Step Brief

PREPARE FIRST. No production connection or agent write/deploy permissions. The data/security owner defines access policy. The engineering manager leads a supervised exercise. The PM protects capacity and validates the business assumptions.

The Decision Brief

PAUSE this proposed use. Review readiness next Tuesday, with access-test evidence and a demonstrated engineering workflow. Then decide whether an approved synthetic-data trial would answer a useful question.

The update to leadership

We propose preparation time to resolve access boundaries and build team readiness. This protects a worthwhile opportunity while making the conditions for responsible delivery explicit. No AI productivity result has been measured.

> They leave with a responsible investment and an accountable next move. The opportunity stays visible, and the prerequisites receive owners.

Take this into the work.

Use this Guide to prepare your judgment, then open your Lead Developer Lab to work your actual situation. Nic's walkthrough demonstrates the same method on a realistic case.

1. Prepare privately

Open Private Prep in the Lead Developer Lab. Write the event, the cost you carry, your Why, the future you could help create, and what you still need to ask your PM.

2. Understand each other and choose a move

Open Conversation. Share the outcome, ask what your counterpart is concerned about, and name what remains unresolved. Export the Shared Next-Step Brief as a proposal or an agreement. Clarifying, asking a specialist, preparing, reviewing evidence, testing, and stopping are all possible next moves.

3. Build a Trial Card when appropriate

Move to Shared Work. Agree on the task, comparator, permission, quality, capacity, bar, stop rule, owners, and review date. Export the Trial Card before the test.

4. Return with evidence

Enter full-workflow effort for both methods, quality results, and comparison limits. Export the Evidence Record. Blank effort stays unknown.

5. Write the decision

Choose Continue, Revise, Stop, or Pause. A conversation or preparation decision can go straight to Decision without a trial. Name the reason, limits, next action, owner, date, and leadership update.

Keep what matters

Shared drafts stay in this browser on this device. Private reflection stays only in the open tab and is not saved or exported. Export shared work before changing devices. There is no cloud synchronization.

> Use Load Examples in each Lab step to see the expected kind of answer. It replaces only that step. Watch Nic work a realistic case to see how the thinking connects.